TASK: 28

(OWASP Mobile Top 10)

Improper Platform Usage

&

Extraneous Functionality

Gabbani Surendra

What is Improper Platform Usage?

Improper Platform Usage refers to incorrect or insecure use of the Android operating system features.This includes misuse of Android components, permissions, intents, WebView, file storage, or security APIs. When the app does not follow Android's security best practices, attackers can exploit these weaknesses to access data, inject intents, or misuse app components.

Examples:

- Exported Activities/Services without protection
- Using insecure storage instead of Android KeyStore
- Declaring unnecessary dangerous permissions
- Using insecure custom WebView configurations
- Not using secure networking APIs
- Misusing Intent filters, allowing unauthorized access

Common mistakes that cause Improper Platform Usage:

Incorrect use of Android components

- Exporting Activities/Services/Broadcast Receivers without protection (exported="true" unintentionally)
- Improper Intent handling

Wrong or excessive permissions

- App requests unnecessary permissions (camera, contacts, SMS)
- Using dangerous permissions without runtime checks

Improper use of WebView

- Enabling JavaScript unnecessarily
- Using add Javascript Interface insecurely

Misusing file storage

- Storing data in external storage instead of internal secure storage

Not using platform security features

- Not using the Keystore for crypto keys

LAB PRACTICAL

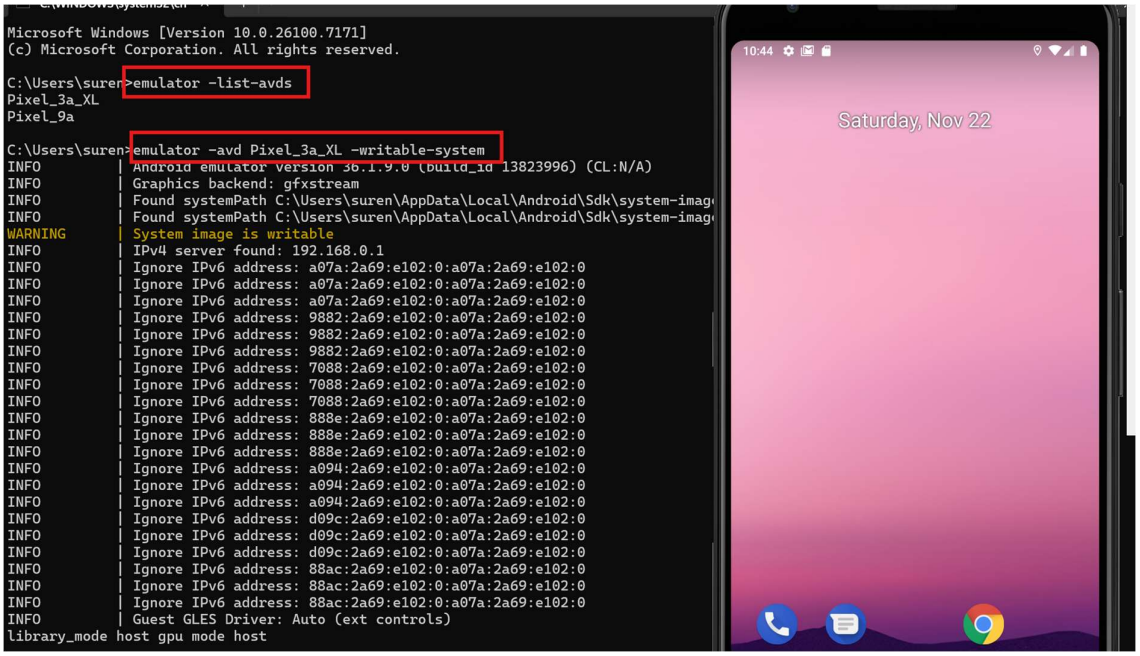
1. Find the Improper platform usage & Extraneous functionality on Lab APK'S (InjuredAndriod & Damn Vulnerable Bank)

APK's:https://github.com/B3nac/InjuredAndroid/releases/download/v1.0.12/InjuredAndroid-1.0.12-release.apk(injured android)

https://github.com/rewanthtammana/Damn-Vulnerable-Bank/blob/master/dvba.apk

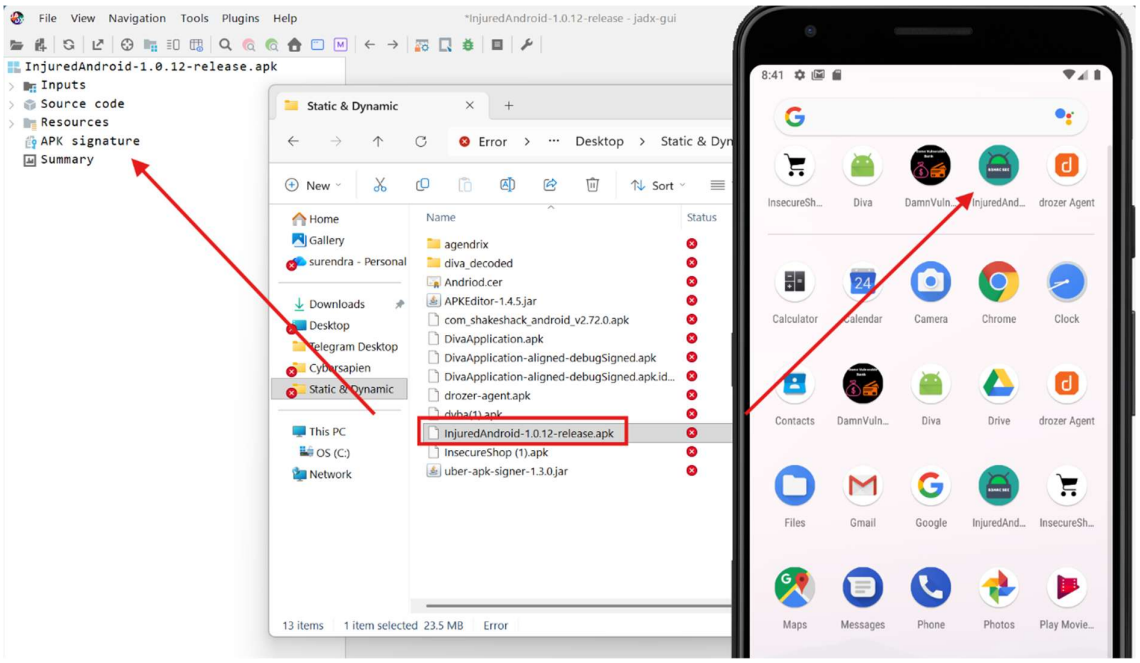
STEP 1:

- Open command prompt and list all the emulators (AVDs).
- Start emulator with writeable system by using commands as below.
- emulator -list-avds.
- emulator -avd Pixel_3a_XL -writable-system.



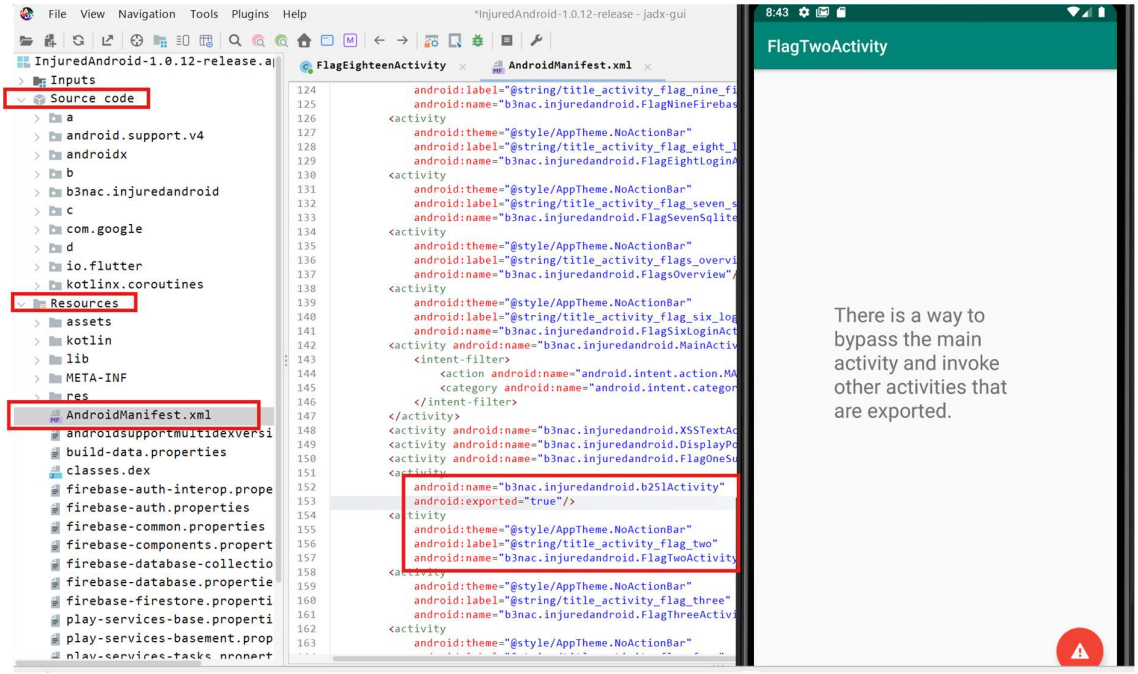
STEP 2:

- Download the APK Application.
- Decompile with the signature.
- Setup the Injured android APK in both jadx and virtual emulator, by simply dragging into.



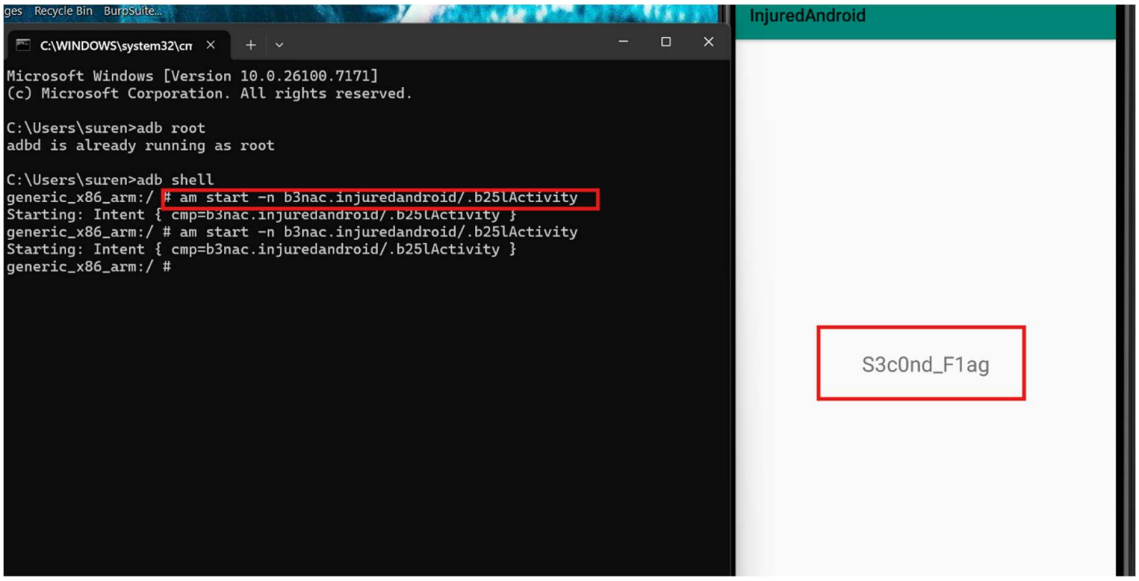
STEP 3:

- View the source code of APK completely.
- Go through the folders of highlighted portion.
- Click andriodmanifest.xml view the source code of Activity 2.
- Here the Activity 2 exported="true".



STEP 4:

- Get root access in emulator and enter Android OS terminal shell.
- Misconfigured exported components in Activity2 by using this commands will redirected to flag in emulator.
- Command" adb shell am start -n b3nac.injuredandroid/.b25lActivity"



Risks of Improper Platform Usage:

- Unauthorized access to app components
- Data leakage through Intents
- Sensitive data exposure
- Malware exploiting poorly configured permissions
- App hijacking or activity injection

Prevention Methods:

- Use least privilege permissions
- Protect exported activities with permissions
- Validate and sanitize input received via Intents
- Use Android Keystore for sensitive data
- Use secure WebView settings

What is Extraneous Functionality?

Extraneous Functionality refers to hidden, leftover, or debugging features that developers accidentally include in the production app.
These functions are not meant for normal users but still exist inside the APK — such as test credentials, admin panels, hidden menus, debug logs, or unused API endpoints

How it occurs?

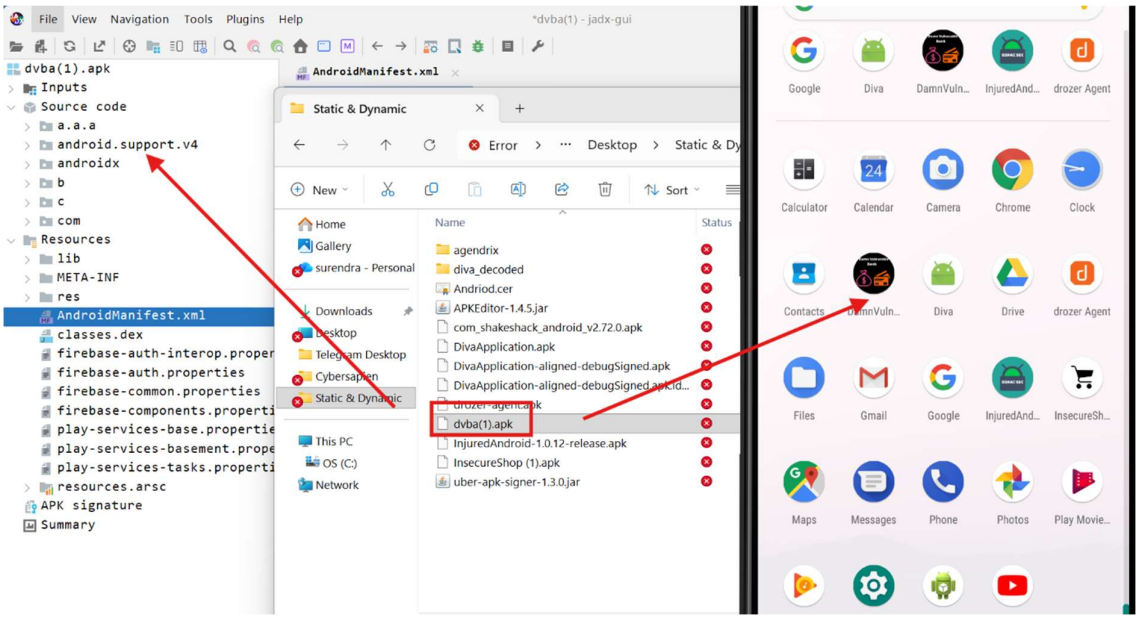
- “Debug” screens still accessible in production
- Uncommented admin bypass logic
- Exposed endpoints like /getAllUsers
- Hardcoded test credentials

LAB

Find the Extraneous Functionality using lab APK, Check if the application is Debuggable/Backup is enabled.

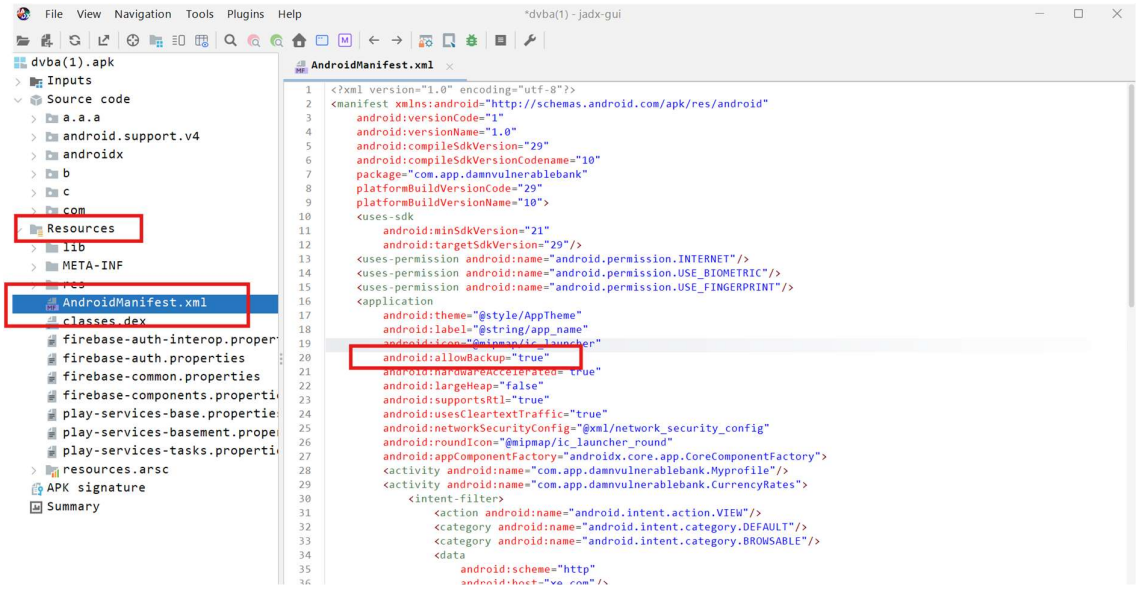
STEP1:

- Setup the Damn vulnerable bank APK in both jadx and virtual emulator, by simply dragging into.
- View android.xml source code.



STEP2:

- Here, debug backup is true, still APK enabling the Backup.



Prevention:

- Remove unused code before release
- Disable developer/debug features
- Apply code obfuscation
- Use secure build configurations
- Perform thorough code reviews & security testing
- Use automated scanning tools (MobSF, QARK)

Impacts of Sensitive Data Exposure:

- Enables unauthorized access to hidden or developer-only features.
- Leaks sensitive information through debug code or logs.
- Allows attackers to bypass authentication or security controls.
- Creates extra attack surfaces through unused or hidden components.
- Facilitates reverse engineering by exposing internal logic or keys.
- Can trigger outdated or insecure backend endpoints

Conclusion:

Improper Platform Usage and Extraneous Functionality both weaken the application's overall security posture by misusing Android platform features and leaving hidden or debug components in the APK. Together, they increase the attack surface, expose sensitive information, and create opportunities for unauthorized access or exploitation.

References:

- https://owasp.org/www-project-mobile-top-10/
- https://mas.owasp.org/
- https://owasp.org/www-project-mobile-security-testing-guide/

THANK YOU